Code Walkthrough for Defense

Marian Ela J. Alibadbad. Algorithmic Optimization of Dense Sphere Packings in Non-Euclidean Spaces.

Purpose: a practice script for explaining three parts of the code line by line: the soft-minimum bound, the polishing step, and why N = 13 is hard. It is written to be read aloud and to be checked against the code in Code_Listing.docx (file algorithms.py). Where a claim comes from the literature and not from this code, it is marked CHECK and should be verified against the cited paper before the defense. Practicing the explanation is the part only you can do: read each section, close the document, and explain the lines from memory, then compare.

1. The soft-minimum energy and its bound

The goal of the Tammes problem is to maximize the smallest distance l_min between any two points. The smallest distance is not smooth (its gradient jumps where two distances tie), so gradient descent cannot use it directly. The code replaces it with a smooth function:

```
E_p = (1/p) ln( sum_k  l_k^(-p) )          (Equation 6 in the paper)
```

where l_k runs over all pair distances and, in the ball problems, the doubled wall clearances 2(R - rho_i). Let m be the number of terms. The bound (Equation 7) says -ln l_min <= E_p <= -ln l_min + (ln m)/p.

Proof, in two lines. The sum contains the term l_min^(-p), and all terms are positive, so the sum is at least l_min^(-p); taking (1/p) ln gives E_p >= -ln l_min. Every term is at most l_min^(-p) (because l_k >= l_min), and there are m terms, so the sum is at most m * l_min^(-p); taking (1/p) ln gives E_p <= (ln m)/p - ln l_min.

Meaning. Minimizing E_p pushes the smallest distance up (E_p is close to -ln l_min), and the gap shrinks as p grows. This is why the code raises p from 2 to 150 during the run: a small p gives a smooth landscape that is easy to descend, a large p gives an energy that is close to the true objective. It also tells you the limit of the method. For N = 13 there are m = 78 pair distances, so at p = 150 the bound allows an error of ln(78)/150 = 0.029 in -ln l_min, which is up to about 3% in l_min. That is an upper bound on the gap between the energy and the objective, not the actual error, but it explains why gradient descent alone does not reach a 0.1% tolerance and why a polishing step helps.

Line by line: _soft_energy (algorithms.py)

```
D = S.pair_dist(X)                  # all pair distances in the space's own
metric
Lp = D[P.iu]                        # upper triangle: each pair once
if P.ball: Lw = max(2*(R - rho), 1e-9); L = concat(Lp, Lw)   # wall
clearances count as distances
L = np.maximum(L, 1e-9)             # avoids log(0) and division by zero
```

```
z = -p * np.log(L)                # z_k = ln( l_k^(-p) )
zm = z.max()                      # largest exponent
w = np.exp(z - zm)                # shifted exponentials, all in (0, 1]
Z = w.sum()
E = (zm + np.log(Z)) / p          # log-sum-exp trick: ln(sum e^z) = zm +
ln(sum e^(z - zm))
```

Why the shift by zm: with p = 150, l^(-p) overflows a floating-point number (for example 0.1^(-150) = 10^150 is fine, but 0.01^(-150) = 10^300 is at the edge). Subtracting the largest exponent before exponentiating gives the same value without overflow. This is the standard log-sum-exp trick.

```
sft = w / Z                        # softmax weights, sum to 1
Wd[P.iu] = -sft[:npair] / L[:npair] # dE/dl_k = -(1/l_k) * softmax_k
G = S.grad_pairs(X, D, Wd)         # chain rule: dl/dx from the geometry
file
if P.ball: wr = 2*sft[npair:]/L[npair:]; G += S.grad_rho(X, wr)   # wall
terms
```

The derivative of E_p with respect to one distance is dE/dl_k = -(1/l_k) * softmax_k. The softmax weights are nearly 1 for the smallest distances and nearly 0 for the others, so the gradient acts almost only on the pairs that currently limit l_min, which is exactly what the max-min problem needs. The geometry file turns dE/dl_k into a gradient with respect to the points. In the tests, this gradient was compared with finite differences and agreed (the difference is below 1e-5).

Line by line: the gradient descent loop (run_gd)

```
frac = pe / budget; p = p0 * (pmax/p0) ** frac     # p grows geometrically
with the share of budget used
E, G, lmin = _soft_energy(P, X, p, True); pe += cost   # one sweep is
charged per evaluation
eta = eta_mult * lmin**2 / p                        # step: curvature of
E_p scales like p / l_min^2
eta = min(eta, 0.25 * lmin / gmax)                  # never move a point
by more than a quarter of l_min
Xn = S.retract(X, -eta * G)                         # move along the
geodesic and stay on the manifold
if En < E: accept, eta_mult *= 1.3  else eta *= 0.4 (at most 5 trials),
eta_mult *= 0.5
```

Be ready to say: the step size is a backtracking rule, not a proven convergence rule. The paper says so (Properties of the Algorithms). Because p changes during the run, the energy being minimized changes too, so there is no theorem that the iterates converge to a local maximum of l_min.

2. The polishing step

After gradient descent has used 85% of the budget (15% in the first study; tuned to 25% in the confirmatory round), the code runs a max-min polishing step. The idea is to treat the problem as what it is, maximize t such that every distance that matters is at least t, and to solve it by repeatedly linearizing.

At the current points, take every pair whose distance is within lcur + 3*eps of the minimum (the 'near-active' pairs). For a small tangent move a_k of each point, the distance between points i and j changes by approximately g_ij . a (a linear function of the moves), where g_ij points along the geodesic joining them. The linear program is:

```
maximize t   subject to   d_ij + g_ij . a  >= t   for all near-active
pairs,
                           2(R - rho_i) + h_i . a >= t   for near-active
wall constraints (ball problems),
                           |a_k| <= eps   for every coordinate (trust
region)
```

The solution a is a move that raises the smallest distance as much as the linearization allows. The code then moves the points (retract), recomputes all distances exactly, and accepts the move only if the exact smallest distance improved. If it improved, the trust region eps grows by 1.5 (at most half of l_min); if not, eps shrinks by 0.4. The loop stops when eps is below 1e-11 or the budget is spent.

Line by line: _polish (algorithms.py)

```
e1, e2 = _tangent_basis(S, X)                  # two orthonormal tangent
directions at every point
ii, jj = where(triu(D < lcur + 3*eps, 1))       # near-active pairs only:
keeps the LP small
uij, uji = _toward(...)                         # unit directions from i
toward j and back
r[2*ii[q]] = inner(e1[ii[q]], uij[q]) ...       # coefficients of the move
of point i in the constraint
r[-1] = 1.0; rhs.append(d[q])                   # row means:  t - g.a <= d
(the LP form of d + g.a >= t)
c[-1] = -1.0                                    # linprog minimizes, so
minimize -t
linprog(c, A_ub, b_ub, bounds=[(-eps, eps)]*(2n) + [(None, None)],
method='highs')
Xn = S.retract(X, V); ln = P.lmin(Xn, Dn)       # exact re-evaluation
decides acceptance
if ln > lcur: accept, eps = min(eps*1.5, 0.5*lcur)  else: eps *= 0.4
```

Things to say honestly. (1) Each polishing iteration is charged one sweep, plus n in the ball problems, but the time to solve the linear program is not charged, so GD+P costs more wall-clock time than its sweep count suggests (about 0.26 s per run more in the first study; see the timing table). (2) Accepting a step only when the exact objective improves makes the best value

monotone, which is why polishing can never make a result worse than where gradient descent stopped (the code also keeps the better of the two). (3) No convergence theorem is claimed. Sequential linear programming is known to converge quickly when the solution is a rigid configuration with enough contacts, which is the typical case here, but this study did not prove it. (4) It is a local method: it improves the basin that gradient descent found and does not search for other basins.

3. Why N = 13 is hard

What the data show (these numbers come from the confirmatory round, 20 fresh seeds, 1,500 sweeps, and are recomputed in conf_summary_by_condition.csv):

```
mean relative score at N = 13:  basin hopping .997 | SA .988 | GD+P .989 |
L-BFGS-B .988 | GD .983 | compression .948
success within 0.1% (relative score >= .999): 0 of 20 seeds for every
method at 1,500 sweeps
across all budgets, only 3 of the 360 runs at N = 13 reached .999 (GD+P
twice at 500 sweeps, SA once at 3,000)
```

So the mean run of the five best methods lands 0.3% to 1.7% below the optimum, which is close, and almost never on it (compression is about 5% below). Why? Three points, in order of how well supported they are.

(a) Evidence from this study: the landscape has many local optima with nearly equal values. For N = 13 the spread of single-run scores in the first study was small (gradient descent .979 to .985, annealing .979 to .996) and the runs end at different configurations, so there is no single basin that attracts almost every start, unlike N = 9 where four of the six methods succeed in most seeds. Basin hopping, which makes jumps between basins, does best, which fits this explanation.

(b) Structure of the problem: 12 points have a perfect arrangement, the icosahedron (minimum angle 63.43 degrees), but a 13th point cannot be added to it without lowering the minimum angle, so the optimum for 13 points (57.1367 degrees, proved by Musin and Tarasov, 2012) is a low-symmetry arrangement that is not a small perturbation of the icosahedron. CHECK the exact description of the optimal configuration in Musin and Tarasov (2012) before stating more than this.

(c) The tolerance: a 0.1% tolerance asks the method to find the optimum to about three digits. A configuration that is in the right basin but not fully relaxed (gradient descent leaves a smoothing bias of up to (ln m)/p, see section 1) still fails. That is why polishing and basin hopping help, and why N = 13 is a demanding test of final precision as well as of search.

What NOT to say: do not say that the algorithms 'cannot' find the optimum, because the data show they can in a few runs, and do not say that curvature or the sphere causes the difficulty. Say that N = 13 has many near-equal local optima and a low-symmetry optimum, that no tested

method reached the 0.1% tolerance reliably, and that this is a result about these implementations at these budgets.

4. Questions to practice (answers are short on purpose)

Q. What is E_p and why not use l_min directly?

A. E_p = (1/p) ln sum l_k^(-p). l_min has no gradient where two distances tie; E_p is smooth and lies within (ln m)/p of -ln l_min.

Q. Prove the bound.

A. The sum is at least its largest term l_min^(-p) and at most m times it. Take (1/p) ln.

Q. Why does p increase during the run?

A. Small p is smooth and easy to descend; large p is close to the true objective. The bound shows the error shrinks as 1/p.

Q. Does your method converge?

A. No convergence theorem is claimed. The paper separates what is proven (the bound) from what is observed (the results).

Q. What does the polishing step do and what does it cost?

A. It solves a linear program for the move that raises the smallest distance most, within a trust region, and accepts only exact improvements. Each iteration is charged one sweep; the LP solve time is not charged, so it costs extra wall-clock time.

Q. Why did you add L-BFGS-B and basin hopping?

A. A reviewer can ask whether gradient descent and annealing are strong baselines. In the confirmatory round, both added methods beat them in all 33 conditions, so the first study's order of those two should be read with that in mind.

Q. What is confirmatory in your study and what is not?

A. Only the second round, whose rules were written and hashed before its fresh seeds were run. The first study is exploratory because the plan was finished after some results were seen.

Q. What does the boundary-share result show?

A. Across 26 ball conditions the density deficit rises with the share of area near the wall (Spearman .87). It does not show that curvature has no effect: that test (C2) was inconclusive.

Q. Which numbers can you not defend?

A. Relative scores for the 16 conditions (of 33) whose reference is the best found in this study (the spherical cap, the hyperbolic disks, and the large hyperbolic balls); they measure distance from our own best, not from the optimum. The other 17 use a proven optimum, a published best-

known value from Sloane, or the Packomania value. I entered the Packomania values after I had seen the first analysis; the decisions were the same with both.

Q. What did the AI do?

A. State it as in your disclosure agreement: it helped write code, analyses, and text; you ran and checked the code and are responsible for the results.

Q. Why a mini-round and what did it show?

A. In the main round the boundary share depends on the radius each method finds, so C1 is partly circular, and C2 on curvature was inconclusive. The mini-round fixes the disk radius and the design boundary share and counts how many disks fit. As preregistered: M1 inconclusive (Spearman .31, p .103), M2 'no detectable curvature effect', M3 described the other spaces (error .028). I WITHDREW the M2 decision: 3 of the 18 conditions were duplicates (see scale-invariance), so the sample was 15 and without them M2 is inconclusive (post hoc). M1 without them is .30, p .143. Say this before anyone asks.

Q. What is scale-invariance and why did it duplicate conditions?

A. The Euclidean plane has no length scale: shrink or stretch a packing and it is still a packing, so disks of radius 0.40 in a ball of radius R are a scaled copy of disks of radius 0.30 in a ball of radius 0.75R. With the same boundary share the Euclidean problem is the same at both radii and gave the same K. The sphere and hyperbolic plane have curvature, so their length scale is fixed and different radii are different problems. So six Euclidean conditions were three, and the model was fitted on three points, not six.

Q. How is K defined and why is it only a lower bound?

A. For a ball and a disk radius r0, K is the largest number n of disks for which a method reaches radius r0 in at least 10 of 20 fresh seeds, scanning n down from the reference count (the whole-space upper bound on density times the ball area, divided by the disk area). Failing to find n disks does not prove they do not fit, so K is a lower bound from the methods used. In the follow-up K is the larger of polished gradient descent and L-BFGS-B; the second method never found more than the first.

Q. Why is the boundary-loss model only a description?

A. It is delta0 = lambda times b0 with lambda fitted on the Euclidean disk. It has one parameter, is not derived from any theory of how a wall disturbs a packing, and the other spaces fit about as well because the deficit follows the boundary share. That shows similarity, not cause, and a different one-parameter curve could fit as well. Follow-up errors: .042 overall, .049 for the cap, close to the .05 limit.

Q. What did the follow-up show?

A. F1 supported (Spearman .59, p .0002) across 33 conditions at fixed radius; F2 inconclusive (hyperbolic term .025, interval -.021 to .070; cap .040, -.005 to .086); F3 described the other

spaces. Its file was hashed before its runs; I report that the files were emailed to my adviser before the runs began.

Q. Did the stronger tuning change the result?

A. I extended the ranges of compression and L-BFGS-B because they sat at the edge. Compression moved inside its range (1.4); L-BFGS-B is still at the edge (2500) but gained about .002. Decisions did not change. Those two methods got nine settings and the others six, so effort is unequal. The original runs are kept and both are reported.

Q. What did saving every configuration show about the wall?

A. One inspected run in the hyperbolic ball of radius 6 had all 50 disks in a ring at the wall. With every configuration saved, the ring (every disk within one disk radius of the wall limit) occurred only in the two largest hyperbolic balls, radii 6 and 8: 225 of the 720 runs there (L-BFGS-B in all of its runs at 1,500 sweeps), and in none of the 8,640 runs in the other 24 ball conditions. So it is a feature of the large hyperbolic balls, not a general result. About a third of all disks touch the wall in every space.

Q. What did the final round show about curvature?

A. 144 equal-area conditions in 48 area-by-count cells, with the cells resampled 10,000 times. With the boundary share in the model the hyperbolic term was .010 (.004 to .017) and the cap term -.009 (-.018 to -.0007). The preregistered rule says no effect when the interval lies within plus or minus .02, so the decision is no detectable curvature effect. Say plainly what that means: no effect as large as .02 of the bound; the intervals do exclude zero, with opposite signs, so about 1% effects are not ruled out and not explained. It concerns the two optimizers at 1,500 sweeps.

Q. What is the rule for the number of disks and how good is it?

A. K_pred = floor(n_ref (1 - 0.46 b0) + 0.5), where n_ref is the count at the upper bound and b0 the boundary share at the fixed radius. The coefficient .46 came from the follow-up and was not refitted. In 20 new conditions it was within one disk every time (exact in 15, mean absolute error .25 disks). Limits: balls of at most 46 disks, K is a lower bound from 20 seeds, and the rule describes the data; it is not derived from theory.

Q. Was the final round time-stamped independently?

A. The file hash and its time were recorded by me. I emailed the files to my adviser before the runs started (06:42 to 06:52 UTC on October 31); the adviser's email is the independent date, and the paper cannot prove it. The Research Plan was finished and hashed before the paper (October 30).

Q. Is your preregistration time-stamped?

A. The hashes were recorded by me, so they are not independent. I emailed the preregistration of the confirmatory round to my adviser on October 28, 2026, before the runs of that round; her

email is the independent record of that file's date. The addendum, the mini-round file, and the follow-up file are preregistered by my hash alone. Be ready to say this plainly.